Pack 02 · Memory and capacity

What fits on a GPU: weights, KV cache, and how many users that leaves room for. Lessons 03–05, 07, 16–19, 25, 42, 50. Answer each question out loud in about a minute, then check the card.

Q1 · Derive KV bytes per token, and what GQA buys SOLID

Lessons: 05, 07, 17

> **INTERVIEWER**
>
> Llama 3 8B has 32 layers, 8 KV heads, head dimension 128, in BF16. How many bytes of KV cache does one token take? Llama 2 7B is similar but has 32 KV heads. What does that do to users per GPU?

60-second answer

- **The formula:** every token stores one K vector and one V vector in every layer, for every KV head, so KV per token = **2 (K and V) × layers × KV heads × head dim × bytes**.
- **Llama 3 8B:** that's 2 × 32 layers × 8 KV heads × 128 × 2 bytes (BF16) = 131,072 bytes, which is **128 KiB** per token.
- **Llama 2 7B:** it has 32 KV heads instead of 8 because it doesn't use GQA, so each token takes 4× as much, **512 KiB**. The same pool then holds a quarter of the tokens, which means about **4× fewer users** per GPU.
- **What GQA buys:** it shares each K/V head across several query heads, so there are fewer KV heads to store. That cuts KV memory and also the KV bytes read on every decode step, with little loss in quality.

Key numbers: 8B: 128 KiB per token; 70B (80 layers, 8 KV heads): 320 KiB per token.

Likely follow-up: "How big is one 4,096-token conversation?" Each token takes 128 KiB, so 4,096 tokens take 4,096 × 128 KiB = 512 MiB, about half a gigabyte for one chat.

> **TRAP**
>
> Name every factor out loud (K and V, layers, KV heads, head dim, bytes). I once gave the exponents without saying what they were.

Q2 · Parameter count and "host MoE like a 13B model" PRIORITY

Lessons: 05, 07

> **INTERVIEWER**
>
> Where do Llama 3 8B's parameters live (h = 4,096, MLP size 14,336, KV width 1,024)? And a teammate says Mixtral 8x7B only uses about 13B parameters per token, so host it like a 13B model on one 80 GB H100 in BF16. Right?

60-second answer

- **Per block:** Q and O are each h × h, K and V are each h × 1,024, and the MLP has three h × 14,336 matrices. That's 2 × 4,096² ≈ 34 M, plus 2 × 4,096 × 1,024 ≈ 8 M, plus 3 × 4,096 × 14,336 ≈ 176 M, for ≈ **218 M** per block. Since 176 ÷ 218 ≈ 0.81, the **MLP is ≈ 81%** of each block.
- **Whole model:** 32 blocks × 218 M ≈ 7 B. The embeddings and the LM head add ≈ 1 B more, which gives the ≈ 8 B total.
- **The MoE rule:** **memory follows total parameters, compute follows active ones**. Every expert has to sit in GPU memory, even though each token only runs through a few of them.
- **Mixtral verdict:** it stores 46.7 B parameters, and at 2 bytes each that's ≈ **93 GB in BF16**, which doesn't fit in 80 GB. So the teammate is wrong: per token it computes like a ≈ 13 B model, which makes it cheap in FLOPs, but it isn't cheap in memory.

Key numbers: block ≈ $2h^2 + 2h \times$ KV width $+ 3h \times$ MLP size.

Likely follow-up: "Does MoE decode read all 93 GB per step?" At large batch, nearly every expert is picked by at least one token in the batch, so yes, each step reads roughly all of it. At batch 1, only the active experts are read.

> **TRAP**
>
> Finish with the verdict. I stated the concept (memory = total) but never said "93 GB doesn't fit in 80 GB".

Q3 · Does Llama 3 70B fit on one H100? PRIORITY

Lessons: 03, 04, 16, 18, 50

> **INTERVIEWER**
>
> Llama 3 70B runs in BF16 across two H100s (80 GB, 3.35 TB/s each). The PM wants it on one. What are the weights in BF16, FP8 and 4-bit, which can actually serve users, and what's the decode time per token?

60-second answer

- **Weights:** weight bytes = parameters × bytes per parameter. At 2 bytes that's ≈ **141 GB** in BF16, at 1 byte it's ≈ **70 GB** in FP8, and at half a byte it's ≈ **36 GB** in 4-bit, including the scale factors.
- **FP8 loads but can't serve:** 70 GB of weights on an 80 GB GPU leaves only about 10 GB, and that has to cover activations, CUDA graphs and the KV cache. So the model loads, but there's almost no room for users.
- **4-bit can serve:** it leaves ≈ 40 GB for the KV cache, and at 320 KiB per token that's room for ≈ 120k tokens.
- **Decode floor:** every decode step reads all the weights, so time per token = weight bytes ÷ bandwidth. BF16 is 141 GB ÷ 3.35 TB/s ≈ 42 ms on one GPU, or ≈ 21 ms split across two. FP8 is 70 ÷ 3.35 ≈ 21 ms, and 4-bit is 36 ÷ 3.35 ≈ **10.7 ms**.

- **Verdict:** FP8 on one GPU already matches BF16 on two at ≈ 21 ms, but it has no room for KV. 4-bit on one GPU fits users and is about twice as fast as BF16 on two (≈ 10.7 ms against ≈ 21 ms).

Key numbers: weights = params × bits ÷ 8; decode floor = weight bytes ÷ bandwidth.

Likely follow-up: "We have A100s. Does FP8 still help?" Partly. FP8 weights still save memory and decode time, because decode time follows the bytes read. But the A100 has no FP8 tensor cores, so the weights are expanded to 16 bits for the math, which means the math runs at 16-bit speed and prefill doesn't speed up.

> **TRAP**
>
> I first said decode time stays the same with fewer bits. It follows the bytes, so it halves with each halving of bits. And it's bytes ÷ (bytes/s), not "bytes per second ÷ bandwidth".

Q4 · "We can only serve 3 users per GPU?" PRIORITY

Lessons: 19, 25, 42

> **INTERVIEWER**
>
> Llama 3.1 8B on one H100, vLLM defaults. The startup log says "Maximum concurrency for 131,072 tokens per request: 3.43x" and the PM panics. How many 1,800-token chats really fit, and what should max_num_seqs be if the p95 request is 2,400 tokens?

60-second answer

- **What the log means:** the 3.43× is a **worst case at max_model_len**, assuming every request grows to 131,072 tokens. The 54.8 GiB pool at 128 KiB per token holds ≈ 449k tokens, and 449k ÷ 131,072 = 3.43, which is exactly the number in the log.
- **Real chats:** at 1,800 tokens each, the same pool holds 449k ÷ 1,800 ≈ **249 at once**, not 3.
- **Setting max_num_seqs:** size it from a **high percentile, not the mean**, because the long requests are the ones that overflow the pool. At the p95 of 2,400 tokens, 449k ÷ 2,400 ≈ 187, and then you **round down** with some margin, to about 160–180.
- **Why round down:** if you admit more requests than fit at their final length, the engine has to push some out partway through, which causes **preemptions**. Those show up as silent tail spikes with zero errors.

Key numbers: pool tokens = pool bytes ÷ KV per token; requests that fit = pool tokens ÷ final tokens per request.

Likely follow-up: "An agent product (1,000-token prompts, 6,000-token outputs) has P99 TPOT spikes but no errors, with max_num_seqs at 1,024 and a 533k-token pool." Each request grows to 1,000 + 6,000 = 7,000 tokens, so only 533k ÷ 7,000 ≈ 76 fit at final length, far below the 1,024 that are admitted. That's why you get preemptions instead of errors, and the counter `num_preemptions_total` confirms it. Set max_num_seqs to ≈ 70, just under the 76 that fit.

> **TRAP**
>
> I confused tokens per request with number of requests ("anything above 2,400"), and wavered on rounding. Always round down: admitting more than fits is what causes preemptions.

Q5 · Three startup crashes, and the utilization cap PRIORITY

Lessons: 18, 42

> **INTERVIEWER**
>
> vLLM crashes on three different days: while loading the model, while sizing the KV cache after setting max_model_len = 1,000,000, and during CUDA graph capture. What ran out each time? Later someone wants to raise gpu_memory_utilization from 0.92 to 0.99. What could go wrong?

60-second answer

- **Loading:** the weights themselves don't fit in GPU memory, so the fix is a smaller precision or more GPUs.
- **Sizing the cache:** with max_model_len = 1,000,000, the KV for one max-length request is bigger than the whole pool. You can't grow the cache, because it already gets everything left over, so the fix is to lower max_model_len.
- **Graph capture:** CUDA graphs need memory from the **headroom outside the budget**, and that headroom is too small. The fix is to lower gpu_memory_utilization or capture fewer graph sizes.
- **A small raise:** going from 0.92 to 0.95 on 79.1 GiB adds $0.03 \times 79.1 \approx 2.4$ GiB to the pool. At 128 KiB per token that's $\approx$ 19k more tokens, which is about 10 more 1,800-token chats.
- **Going to 0.99:** that leaves almost no headroom, so it risks an **out-of-memory crash at runtime under peak load**, even after a clean startup.

Key numbers: extra KV tokens = Δutilization $\times$ GPU memory $\div$ KV per token.

Likely follow-up: "Why is the runtime failure worse than a startup failure?" A startup failure shows up straight away. The runtime one passes every test at low load, so it ships, and then it crashes in production at the peak.

> **TRAP**
>
> For crash 2 I said "reduce the KV cache". The max length is what's too big. For 0.99 I named the headroom but not when it fails (at runtime).

Q6 · fp8 KV cache or 8-bit weights? PRIORITY

Lessons: 42, 50

> **INTERVIEWER**
>
> Same 8B server: 54.8 GiB pool, 128 KiB per token, 15 GiB of weights. You can flip one flag: fp8 KV cache, or 8-bit weights (15 → 7.5 GiB). Which adds more KV capacity, and does the answer change for 70B on two H100s?

60-second answer

- **fp8 KV cache:** it **halves bytes per token**, from 128 KiB to 64 KiB, so the same pool holds twice as many tokens. That **doubles** capacity, from 449k to ≈ 900k tokens.
- **8-bit weights:** they **add a fixed amount**, equal to the memory they free. Going from 15 to 7.5 GiB frees 7.5 GiB, and 7.5 GiB ÷ 128 KiB ≈ 61k extra tokens.
- **For 8B, fp8 KV wins easily:** the pool is large and the weights are small, so doubling the pool adds ≈ 449k tokens while shrinking the weights adds only ≈ 61k.
- **For 70B on two H100s, the answer flips:** the weights (141 GB) dominate and the pool is small, so doubling it doesn't add much. 8-bit weights free ≈ 70 GB, which is far more than doubling the small pool would give, so **8-bit weights win**.

Key numbers: fp8 KV: capacity × 2; weight quantization: + freed bytes ÷ KV per token.

Likely follow-up: "Can you do both?" Yes, because they save memory in different places, and on 70B that's the usual setup: FP8 weights and an FP8 KV cache.

> **TRAP**
>
> I said both flags double capacity. Only fp8 KV doubles it; smaller weights add a fixed amount.